

Guía de Claude Code

Trabajar en local — Javy Suplementos

Tienda de suplementos · San Miguelito, Panamá

1. Arranque diario (rutina)

```
cd javysuplementos
code .           # abre VS Code
claude           # inicia Claude Code en la terminal
git pull origin claude # trae lo último (o usa /sync)
```

Para ver los cambios en vivo escribe `/preview` y abre la URL (normalmente `http://localhost:8080`).

2. Comandos slash (atajos a medida)

Escribe `/` en Claude y aparecen. Los de este proyecto:

Comando	Qué hace
<code>/preview</code>	Levanta un servidor local para ver el sitio en vivo.
<code>/sync</code>	Pull + push seguro de la rama <code>claude</code> (nunca toca <code>main</code>).
<code>/nuevo-producto</code>	Agrega un producto sincronizando Supabase y <code>product-data.js</code> .
<code>/optimizar-img</code>	Convierte PNG pesados a WebP y actualiza referencias.
<code>/revisar</code>	Revisión paralela de diseño + lógica de tus cambios.

Puedes pasarle datos, por ejemplo:

```
/nuevo-producto Proteína Whey Gold marca ON, $54, 5lb, chocolate y vainilla
```

Para crear uno nuevo: crea un `.md` en `.claude/commands/`. El nombre del archivo es el nombre del comando.

3. Subagentes (varias cosas a la vez)

Ayudantes especializados que corren **en paralelo**. Este proyecto trae dos:

- **diseñador** — audita diseño, CSS y responsive (mobile/tablet/desktop).
- **logica** — audita JS, funcionalidad y seguridad (escapeHTML, WhatsApp...).

Cómo usarlos

- **Automático:** "Revisa el diseño y la lógica de los últimos cambios a la vez."
- **Explícito:** "Usa el subagente *diseñador* para revisar product.css."
- **Con `/revisar`** — lanza ambos a la vez y consolida hallazgos.

💡 La clave para paralelizar: pide varias cosas independientes en **un mismo mensaje**. Claude lanza varios agentes a la vez.

4. Permisos (menos interrupciones)

`.claude/settings.json` ya autoriza los comandos seguros (git, server local, lectura, `cwebp`) para que no te pregunte a cada rato. Lo que sigue protegido:

- `git push origin main` está **bloqueado** (producción solo por PR).
- No lee secretos (`.env`, `.key`, `.pem`).

Para ajustarlos usa el comando `/permissions`.

⚠ Existe `--dangerously-skip-permissions` (saltar todos los permisos). NO lo uses salvo en entornos aislados: quita los frenos de seguridad.

5. Plan Mode (cambios grandes)

Presiona `Shift+Tab` para ciclar modos hasta "plan mode". Ahí Claude **investiga y propone un plan** sin tocar nada; tú apruebas y recién ejecuta. Ideal para rediseños o cambios que tocan varios archivos.

6. Varias features a la vez — Git Worktrees

Un *worktree* es una segunda carpeta con otra rama del mismo repo. Te deja tener dos features abiertas a la vez, cada una con su sesión de Claude, sin pisarse.

```
# Crear un worktree para una feature nueva
git worktree add ../javy-footer -b feature/footer

# Abrirlo en otra ventana de VS Code y correr Claude ahí
cd ../javy-footer
claude

# Al terminar
git worktree remove ../javy-footer
```

Alternativa simple: abrir 2–3 terminales con `claude` en la misma carpeta, para tareas que no chocan entre sí.

7. Buenas prácticas del proyecto

- Siempre se trabaja en la rama **claude**. A `main` solo por PR aprobado.
- Imágenes nuevas en **WebP** cuando se pueda.
- Texto de usuario/BD al DOM: siempre con `escapeHTML()`.
- El número de WhatsApp solo vive en `js/whatsapp-config.js`.
- Productos: sincronizar Supabase y `js/product-data.js`.

8. Chuleta rápida

Quiero...	Hago...
Ver el sitio en vivo	<code>/preview</code>
Bajar/subir cambios seguro	<code>/sync</code>
Agregar un producto	<code>/nuevo-producto ...</code>
Aligerar imágenes	<code>/optimizar-img</code>
Revisar diseño + lógica	<code>/revisar</code>
Planear un cambio grande	<code>Shift+Tab</code> → Plan Mode
Ajustar permisos	<code>/permissions</code>
Ver/crear subagentes	<code>/agents</code>
Dos features a la vez	<code>git worktree add ...</code>